

COMP9001

Lecture 11

Introduction to Programming

Presenter: Dr Armin Chitizadeh

Version 0.1



THE UNIVERSITY OF
SYDNEY

Inspired by Hazem El-Alfy 2024 slides

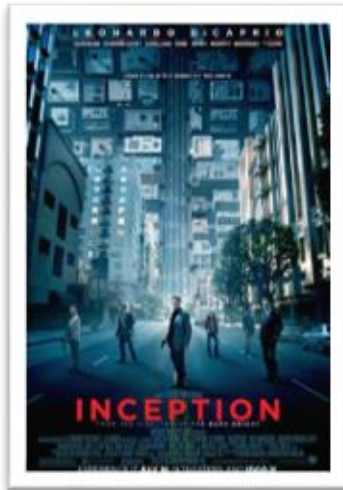
CRICOS 00026A TEQSA PRV12057



We recognise and pay respect to the Elders and communities – past, present, and emerging – of the lands that the University of Sydney's campuses stand on. For thousands of years they have shared and exchanged knowledges across innumerable generations for the benefit of all.



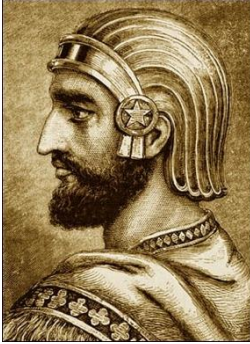
Recursion



Why are We Learning **Recursion** Technique?

- Many problems that can be done with recursion can be done with iteration
 - But for some, recursion can be way easier
- It is a common technique in AI
- It differentiate great programmer against normal programmer
- With some modifications, it can be extremely efficient (will be covered in other units)

Chapar Khaneh (Postal Service System)



Chapar Khaneh (Postal)



Chapar Khaneh (Postal)

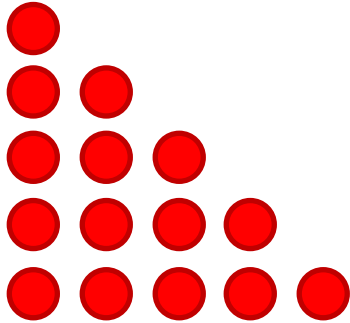


Let's summarise what you saw on the last slide:

- Each postman had to wait and be paid for the next postman to arrive
- As soon as they returned the information, their contract was finished
- Each postman, called another postman
- They used what the next postman said to generate their answer

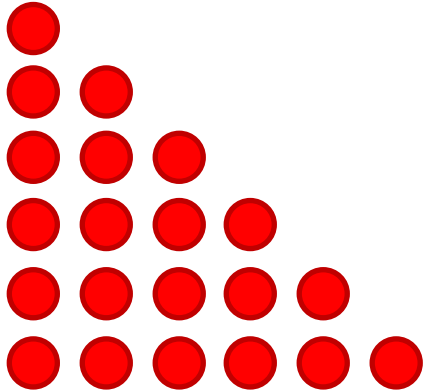
Calculating the sum of the first n natural numbers

(n = 5)



Calculating the sum of the first n natural numbers

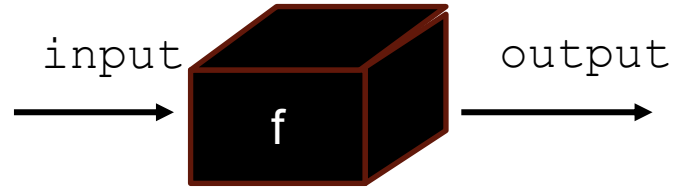
(n = 6)



$$\text{sum}(6) = \text{sum}(5) + 6$$

$$\text{sum}(n) = \text{sum}(n-1) + n$$

Remember Functions?



$$\begin{aligned} \text{sum}(n) = \\ \text{sum}(n-1) + n \end{aligned}$$

What is Recursion?

- A function that calls itself is **recursive**.
- The process of executing is called **recursion**.

What is Recursion?

- Whenever you solve a **big problem** by cheating and using the solution to a **smaller problem** to get to the solution to your **big problem**.

- *Professor Richard Buckland*

- Recursion is a problem-solving technique
 - Solving a problem by solving smaller version of itself

What is Recursion?



Demo!

Calculating the sum of the first n natural numbers

$$\text{sum}(6) = \text{sum}(5) + 6$$

$$\text{sum}(n) = \text{sum}(n-1) + n$$

Recursion Parts



- Base case:

- The simplest case
 - Also known as stopping condition

- Recursion relation:

- a mathematical equation that defines a sequence by relating each term to one or more previous terms in the sequence
- In each recursion relation there is a recursive call:
 - Recursive call is when a function calls the same function but with different arguments

```
1 #####
2 ## Calculating the sum of the first n natural numbers
3 #####
4 #####
5
6 def sum_natural(n):
7     if n == 1:
8         return 1
9     else:
10        print(f"n:{n}")
11        answer = sum_natural(n-1) + n
12        return answer
13
14
15 print(f"sum_natural(6): {sum_natural(6)}")
```

Recursive Call

```
1 #####
2 ## Calculating the sum of the first n natural numbers
3 #####
4 #####
5
6 ▼ def sum_natural(n):
7 ▼     if n == 1:
8         return 1
9 ▼     else:
10         answer = sum_natural(n-1) + n
11         return answer
12
13
14 print(f"sum_natural(3): {sum_natural(3)}")
```



A red arrow originates from the line `answer = sum_natural(n-1) + n` in the `sum_natural(3)` call and points to the `def sum_natural(3):` block.

```
def sum_natural(3):
    if 3 == 1:
        return 1
    else:
        answer = sum_natural(3-1) + 3
        return answer
```



A red arrow originates from the line `answer = sum_natural(3-1) + 3` in the `sum_natural(3)` block and points to the `def sum_natural(2):` block.

```
def sum_natural(2):
    if 2 == 1:
        return 1
    else:
        answer = sum_natural(2-1) + 2
        return answer
```



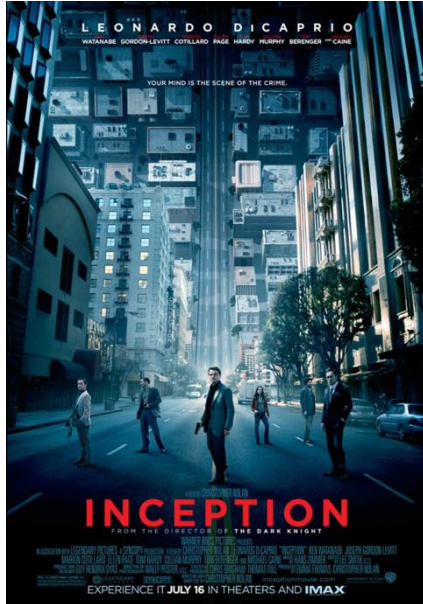
A red arrow originates from the line `answer = sum_natural(2-1) + 2` in the `sum_natural(2)` block and points to the `def sum_natural(1):` block.

```
def sum_natural(1):
    if 1 == 1:
        return 1
    else:
        answer = sum_natural(1-1) + 1
        return answer
```

Different Ways to Implement the Base Case in Python

```
1 #####
2 ## Calculating the sum of the first n natural numbers
3 #####
4 #####
5
6 ▼ def sum_natural(n):
7 ▼     if n == 1:
8         return 1
9 ▼     else:
10         answer = sum_natural(n-1) + n
11         return answer
12
13
14 print(f"sum_natural(3): {sum_natural(3)}")
```

Different Ways to Implement the Base Case in Python



Different Ways to Implement the Base Case in Python

- Let's write different variants of base-case for `sum_natural` recursive function

More Examples

Counter

- Using the recursion write a program that prints from n to 1
- Can you make it print in reverse with minimal change?

One common mistake

Infinite Recursion

Infinite Recursion

- Sequence of calls must reach its base case, otherwise it goes on making recursive calls forever (*infinite recursion*)



Recursive Call

```
1 #####
2 ## Calculating the sum of the first n natural numbers
3 #####
4 #####
5
6 ▼ def sum_natural(n):
7 ▼     if n == 1:
8         return 1
9 ▼     else:
10         answer = sum_natural(n-1) + n
11         return answer
12
13
14 print(f"sum_natural(3): {sum_natural(3)}")
```



```
def sum_natural(3):
    if 3 == 1:
        return 1
    else:
        answer = sum_natural(3-1) + 3
        return answer
```



```
def sum_natural(2):
    if 2 == 1:
        return 1
    else:
        answer = sum_natural(2-1) + 2
        return answer
```



```
def sum_natural(1):
    if 1 == 1:
        return 1
    else:
        answer = sum_natural(1-1) + 1
        return answer
```

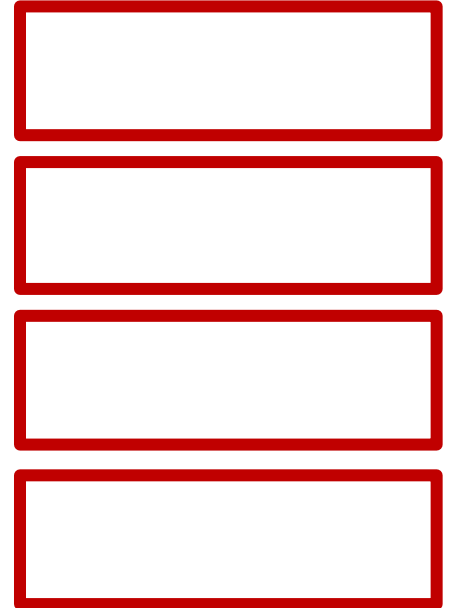
Issue with Recursive Calling

memory usage

Issue with Recursive Calling

memory usage

```
1 #####
2 ## Calculating the sum of the first n natural numbers
3 #####
4 #####
5
6 ▼ def sum_natural(n):
7 ▼     if n == 1:
8         return 1
9 ▼     else:
10         answer = sum_natural(n-1) + n
11         return answer
12
13
14 print(f"sum_natural(3): {sum_natural(3)}")
```



Issue with Recursive Calling memory usage

```
1 #####
2 ## Calculating the sum of the first n natural numbers
3 #####
4 #####
5
6 ▼ def sum_natural(n):
7 ▼     if n == 1:
8         return 1
9 ▼     else:
10         answer = sum_natural(n-1) + n
11         return answer
12
13
14 print(f"sum_natural(3): {sum_natural(3)}")
```

sum_natural

$n \rightarrow 1$

sum_natural

$n \rightarrow 2$

sum_natural

$n \rightarrow 3$

__main__

So Why Are We Using Recursion?

Let's do an experiment!

- Finding the max value in a binary tree
- Vs
- Finding the max number in a list

- We need volunteers:
 - 15 volunteers to return the max number, given 3 numbers
 - 1 volunteer to return the max number, given 15 numbers
 - 1 volunteer to live stream the event to echo360. (*Tools provided*)

So Why Are We Using Recursion?

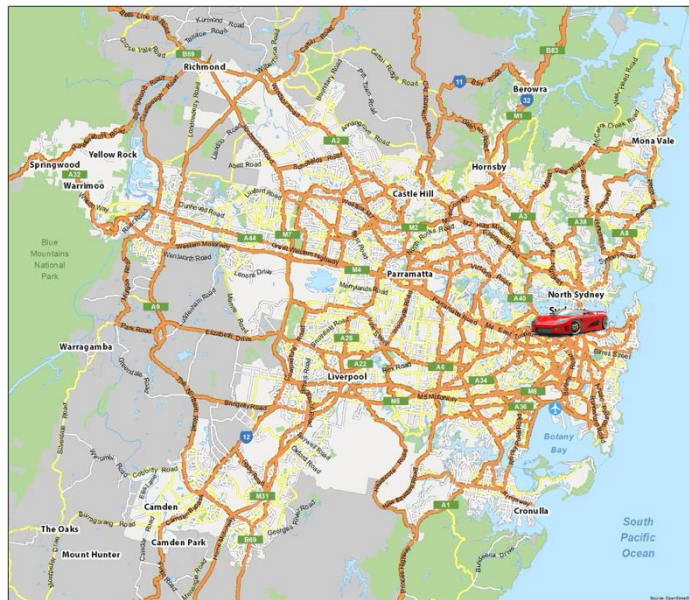
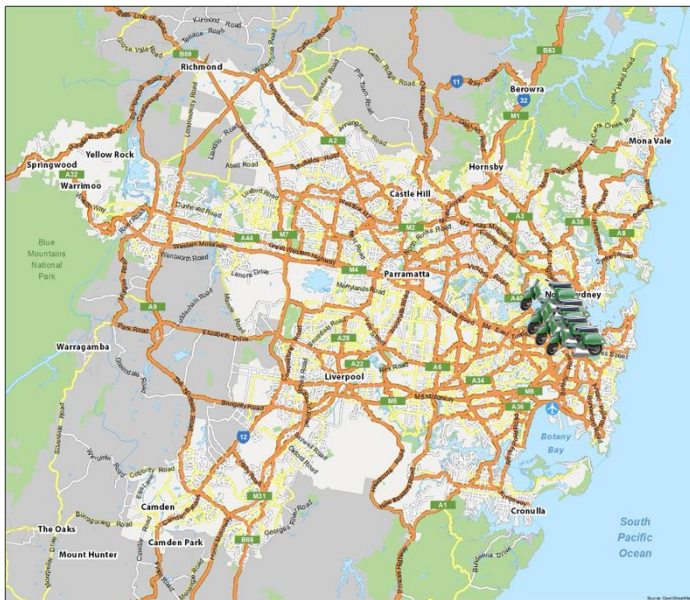
Binary Tree Max Finder

```
1 class TreeNode:
2     def __init__(self, value, left=None, right=None):
3         self.value = value
4         self.left = left
5         self.right = right
6
7     def find_max(node):
8         if node is None:
9             return -100
10
11         left_max = find_max(node.left)
12         right_max = find_max(node.right)
13
14         return max(node.value, left_max, right_max)
15
```

```
16 if __name__ == "__main__":
17     # These structures will not be asked in the exam
18     # Constructing a full binary tree with 15 nodes
19     # Tree structure:
20     #           10
21     #        /  \
22     #       32   14
23     #      / \   / \
24     #     53 78 12 66
25     #    /\ /\ /\ /\
26     #   21 41 71 92 81 93 75 17
27
28     root = TreeNode(10,
29                     left=TreeNode(32,
30                                 left=TreeNode(53,
31                                                left=TreeNode(21),
32                                                                right=TreeNode(41)),
33                                 right=TreeNode(78,
34                                                left=TreeNode(71),
35                                                                right=TreeNode(92))),
36                     right=TreeNode(14,
37                                    left=TreeNode(12,
38                                                    left=TreeNode(81),
39                                                                right=TreeNode(93)),
36                                    right=TreeNode(66,
37                                                    left=TreeNode(75),
38                                                                right=TreeNode(17)))
39
40     max_value = find_max(root)
41     print("Maximum value in the tree:", max_value)
42     # Can you tell in which orders nodes are visited?
```




5 slow motorcycles vs 1 fast car
Who would win?



More Examples

Calculate the sum of items in a list using recursion

More Recursion Examples

Tower of Hanoi

Let's Play the Game!

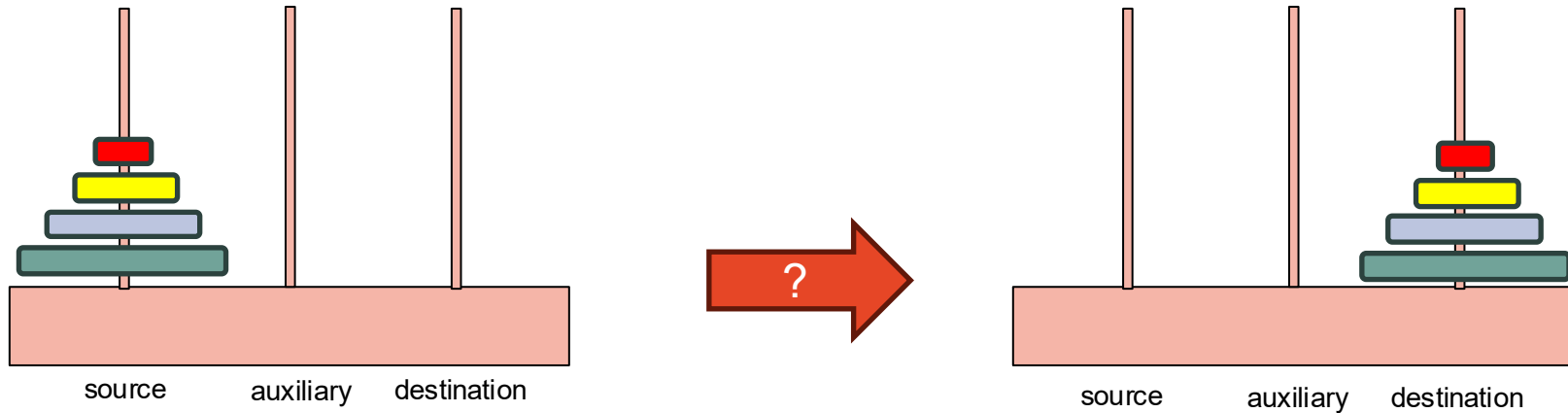
<https://www.mathsisfun.com/games/towerofhanoi.html>



Etsy [Link](#)

More Recursion Examples

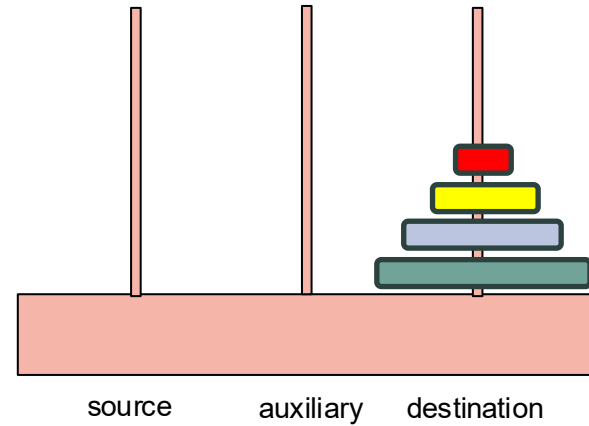
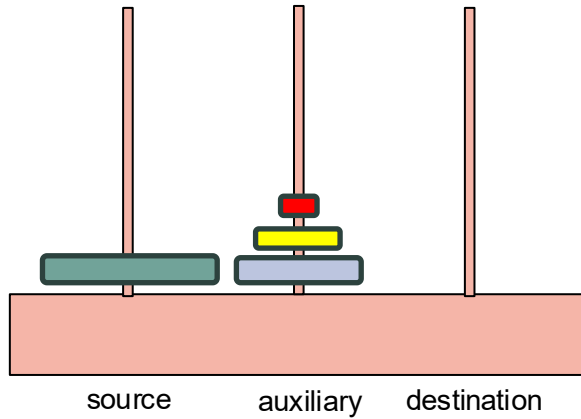
Tower of Hanoi



The Tower of Hanoi puzzle involves moving a stack of disks from source rod to destination rod, following specific rules. You can only move one disk at a time, and a larger disk cannot be placed on top of a smaller disk. The goal is to move all disks from the starting rod to a destination rod, using an auxiliary rod as needed.

More Recursion Examples

Tower of Hanoi



More Recursion Examples You Choose!



Geeks for Geeks Recursion Interview Questions

- <https://www.geeksforgeeks.org/top-50-interview-problems-on-recursion-algorithm/>

- Vote!

- <https://www.menti.com/alne5ocvsv61>



Thanks you!

Fully anonymous feedback form
for COMP9001 students



<https://forms.office.com/r/4Q0die7fFK>

Do you know what this is?

